ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΘΕΜΑΤΑ ΕΠΙΣΤΗΜΗΣ ΔΕΔΟΜΕΝΩΝ

ΑΝΔΡΙΑΝΟΠΟΥΛΟΣ ΝΙΚΟΛΑΟΣ | Π20018

ΚΡΙΠΑΡΑΚΟΣ ΠΑΝΑΓΙΩΤΗΣ | Π20106

Πειραιάς, *Φεβρουάριος 2024*

Σύνολο Δεδομένων & Στόχος της εργασίας

Το σύνολο δεδομένων πάνω στο οποίο δουλέψαμε είναι το PEMS-BAY. Πρόκειται για ένα dataset που περιλαμβάνει δεδομένα και πληροφορίες που έχουν συλλεχθεί από 325 αισθητήρες στο οδικό δίκτυο της Bay Area, California. Στην διάθεσή μας έχουμε δύο αρχεία τύπου CSV. Το αρχείο PEMS-BAY.csv περιέχει τα δεδομένα που κατέγραψαν οι αισθητήρες για 6 μήνες, από 01-01-2017 00:00 μέχρι 30-06-2017 23:55. Στο διάστημα αυτό αισθητήρες σημείωναν νέα εγγραφή κάθε 5 λεπτά. Επίσης, υπάρχει στη διάθεσή μας ένα αρχείο μεταδεδομένων (.pkl) καθώς και το PEMS-BAY-META.csv τα οποία περιέχουν πληροφορίες για κάθε έναν από τους 325 αισθητήρες. Στόχος της συγκεκριμένης έρευνας είναι αφού γίνει η απαραίτητη μελέτη και προεπεξεργασία του συνόλου δεδομένων που διαθέτουμε, να εφαρμοστούν τεχνικές μηχανικής μάθησης με στόχο την πρόβλεψη δεδομένων μίας ώρας. Τα αποτελέσματα, θα αξιολογηθούν χρησιμοποιώντας τις μετρικές MSE, RMSE, MAPE & R^2.

Οργάνωση αρχείων

Τα αρχεία της εργασίας είναι τοποθετημένα σε φακέλους. Στον φάκελο data υπάρχουν τα csv αρχεία που περιέχουν την πορεία των δεδομένων μέσα από τα στάδια της προεπεξεργασίας και feature engineering. Στον φάκελο notebooks υπάρχουν τα αρχεία jupyter notebook (.ipynb) τα οποία είναι αριθμημένα και περιέχουν τα 4 βήματα της εργασίας ξεχωριστά. Στον φάκελο plots υπάρχουν τα plot που δημιουργήθηκαν κατα την εκτέλεση του 3ου ερωτήματος.

Λόγω μεγάλου όγκου παραδοτέου και ταυτόχρονου περιορισμού της πλατφόρμας Gunet2 στον φάκελο συμπεριλάβαμε μόνο τα notebook και τα plot, ενώ στον σύνδεσμο εδώ υπάρχει όλη η εργασία στο Google Drive, εδώ.

Οργάνωση κεφαλαίων εργασίας | Περιεχόμενα

I. Data Preprocessing……..2

II. Feature Engineering……..5

III. Trend/Seasonal Behavior……5

IV. Forecasting model……8

V. Παράρτημα κώδικα...……..11

VI. Βιβλιογραφία……13

I. Data Preprocessing

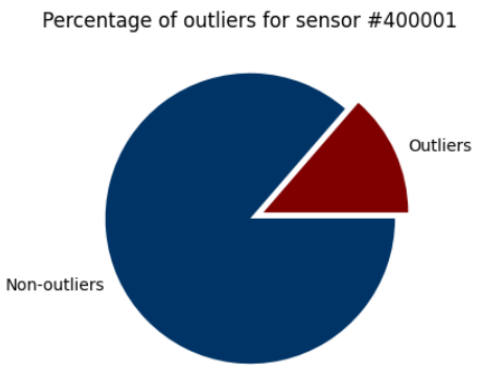
Αρχικά, αφού εισάγαμε τα δεδομένα του κύριου csv αρχείου (PEMS-BAY.csv) στον κώδικα μας σε μορφή dataframe με την χρήση της βιβλιοθήκης pandas, παρατηρήσαμε πως η στήλη του που περιέχει τα timestamp δεν έχει όνομα και το notebook της χαρακτηρίζει ως “Unnamed:0”. Έτσι, με την χρήση της συνάρτησης rename καταφέραμε να της δώσουμε ένα αντίστοιχο όνομα με το περιεχόμενό της, ονομαζοντάς την “timestamp”. Έπειτα, με την χρήση της set_index ορίσαμε αυτήν την στήλη ώς index για το dataframe. Επόμενο βήμα της προεπεξεργασίας δεδομένων είναι ο έλεγχος για null τιμές ή αλλιώς κενά κελιά στο dataframe. Με την χρήση των κατάλληλων εντολών αποδείχθηκε πως δεν υπάρχουν κενά κελιά στο dataframe οπότε μπορούμε να μεταβούμε στην διαδικασία εντοπισμού ακραίων τιμών και θορύβου.

timestamp	400001	400017	400030	400040	400045	400052	400057	400059	400065	400069
2017-01-01 00:00:00	71.4	67.8	70.5	67.4	68.8	66.6	66.8	68.0	66.8	69.0
2017-01-01 00:05:00	71.6	67.5	70.6	67.5	68.7	66.6	66.8	67.8	66.5	68.2
2017-01-01 00:10:00	71.6	67.6	70.2	67.4	68.7	66.1	66.8	67.8	66.2	67.8
2017-01-01 00:15:00	71.1	67.5	70.3	68.0	68.5	66.7	66.6	67.7	65.9	67.8
2017-01-01 00:20:00	71.7	67.8	70.2	68.1	68.4	66.9	66.1	67.7	66.1	67.8

5 rows × 325 columns

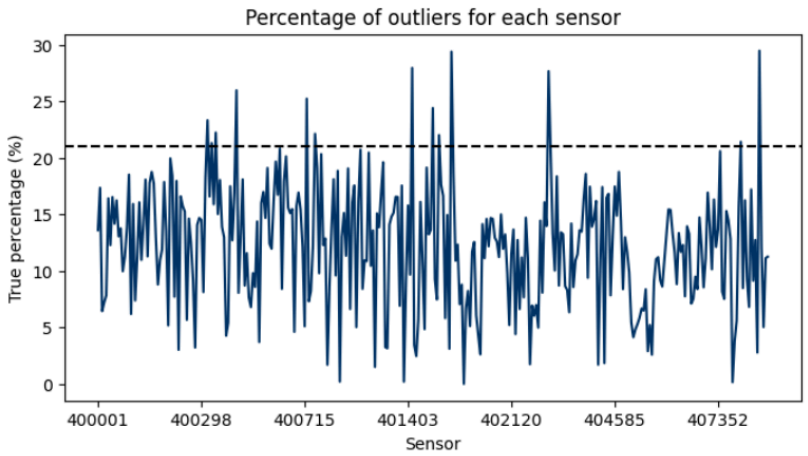
Εικόνα 1 | αρχικό dataframe

Για την διαδικασία της εύρεσης των ακραίων τιμών ακολουθήσαμε την τεχνική Turkey’s Fences. Πρόκειται για την χρήση της interquantile μεθόδου (IQR) με ένα συγκεκριμένο threshold. Το threshold που συνήθως χρησιμοποιείται είναι 1.5. Όσο αυξάνεται αυτή η τιμή, μειώνονται οι ενδεχόμενες ακραίες τιμές. Αντίστοιχα, όσο μειώνεται το threshold, αυξάνεται το πλήθος των ακραίων τιμών. Για λόγους της συγκεκριμένης εργασίας χρησιμοποιήσαμε το threshold 1.5. Το αποτέλεσμα ήταν ένα dataframe ίδιου μήκους με το αρχικό με την διαφορά ότι στην αντίστοιχη θέση της κάθε τιμής τοποθετείται μία boolean τιμή ανάλογα με το αν θεωρείται ακραία τιμή ή όχι (π.χ. έστω ότι για τον αισθητήρα 400001 στις 02-02-2017 00:20 η τιμή θεωρείται outlier, τότε στο ίδιο κελί τοποθετείται η τιμή True).



Εικόνα 2 | 13.61% outliers για τον #400001

Στο παρακάτω line plot βλέπουμε πως το ποσοστό των outlier αυξομειώνεται από αισθητήρα σε αισθητήρα με τους περισσότερους να κυμαίνονται μεταξύ του 7% και του 21% ενώ σε ελάχιστες περιπτώσεις το ποσοστό αυτό φτάνει και ξεπερνά το 25%.



Εικόνα 3 | ποσοστό ακραίων τιμών ανά αισθητήρα

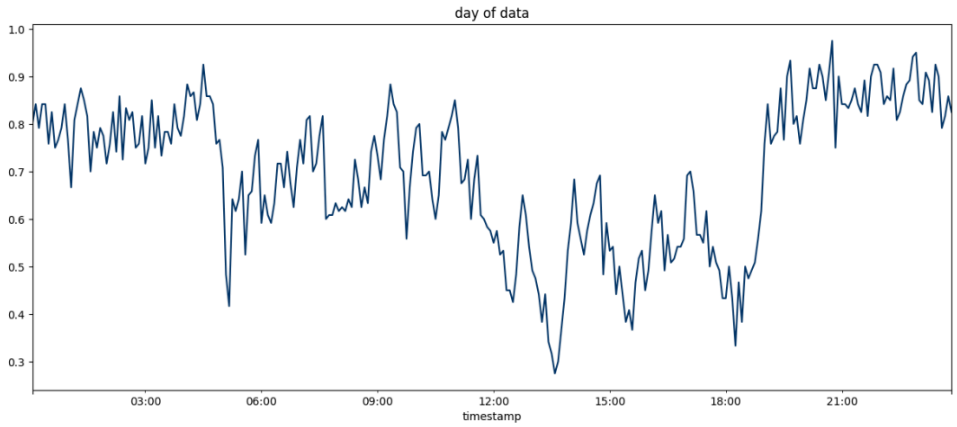
Με την διαγραφή των εγγραφών που περιέχουν ακραίες τιμές το dataset θα μειωθεί υπερβολικά, κάνοντας έτσι κάθε μελλοντική ανάλυση και μελέτη μη ρεαλιστική και κατ' επέκταση λάθος. Έτσι αποφασίσαμε να αντικαταστήσουμε τις ακραίες τιμές με την διάμεση (median) τιμή κάθε στήλης ούτως ώστε να προσπαθήσουμε να προσεγγίσουμε την μελλοντική ανάλυση με ένα πλήρης σύνολο δεδομένων. Επόμενο και τελευταίο βήμα της προεπεξεργασίας είναι η κανονικοποίηση των αριθμητικών δεδομένων. Αρχικά χρησιμοποιήσαμε τον StandardScaler της βιβλιοθήκης sklearn, όμως διακρίναμε μεγάλη απόκλιση ανάμεσα στις κανονικοποιημένες τιμές (από 0.002 έως 1.42). Επόμενο βήμα ήταν η δοκιμή του MinMaxScaler της ίδιας βιβλιοθήκης με τον οποίο είδαμε καλύτερα αποτελέσματα καθώς κανονικοποιεί τις τιμές του dataframe στην κλίμακα [0,1].

timestamp	400001	400017	400030	400040	400045	400052	400057	400059	400065	400069
2017-01-01 00:00:00	0.841667	0.709302	0.953704	0.822581	0.813889	0.716216	0.855932	0.887755	0.818182	0.923077
2017-01-01 00:05:00	0.858333	0.709302	0.956790	0.826613	0.811111	0.716216	0.855932	0.867347	0.790909	0.879121
2017-01-01 00:10:00	0.858333	0.709302	0.944444	0.822581	0.811111	0.648649	0.855932	0.867347	0.763636	0.857143
2017-01-01 00:15:00	0.816667	0.709302	0.947531	0.846774	0.805556	0.729730	0.847458	0.857143	0.736364	0.857143
2017-01-01 00:20:00	0.866667	0.709302	0.944444	0.850806	0.802778	0.756757	0.826271	0.857143	0.754545	0.857143

5 rows × 325 columns

Εικόνα 4 | κανονικοποιημένα δεδομένα

Η προβολή των κανονικοποιημένων δεδομένων είναι δύσκολη για 325 αισθητήρες ταυτόχρονα. Έτσι δημιουργήσαμε μια συνάρτηση με την χρήση της οποίας προβάλλονται σε ένα plot τα δεδομένα που σύλλεξε ο ένας (προκαθορισμένος αισθητήρας) για μια (προκαθορισμένη) ημέρα.



Εικόνα 5 | loop detector #400001 25η Μαρτίου 2017

Έπειτα, εισάγαμε στο notebook το αρχείο που περιέχει τα στοιχεία του κάθε αισθητήρα (PEMS-BAY-META.csv). Αφού κάναμε τον τυπικό έλεγχο για ελλιπείς εγγραφές παρατηρήσαμε πως σε 27 σειρές υπάρχουν κενά, συγκεκριμένα και τα 27 στην στήλη 'City'. Επειδή ο κωδικός της κάθε πόλης εξαρτάται άμεσα από τις συντεταγμένες των συνόρων, (στην συγκεκριμένη περίπτωση από τις συντεταγμένες των αισθητήρων) και ταυτόχρονα παρατηρήσαμε πως για αρκετούς αισθητήρες είναι ίδιος πριν και μετά την εμφάνιση κενού κελιού οπότε αποφασίσαμε να καλύψουμε αυτά τα κενά με την μέθοδο front-fill, βάζοντας στην θέση του κενού κελιού την τιμή του ακριβώς προηγούμενου. Ενδεχομένως να υπάρχουν ελάχιστες αστοχίες και να έχει χρησιμοποιηθεί λάθος κωδικός πόλης για ελάχιστους από τους 27 αισθητήρες, πράγμα που δεν επηρεάζει δραστικά

οποιαδήποτε μελλοντική ανάλυση. Προβάλλοντας τα δεδομένα του δεύτερου αρχείου (PEMS-BAY-META.csv) είδαμε πως πρόκειται για ευαίσθητα δεδομένα όπως οι γεωγραφικές συντεταγμένες, η πόλη, η οδός, το πλήθος των λωρίδων και η ονομασία του αισθητήρα. Τα δεδομένα αυτά δεν υπόκεινται σε κάποια προεπεξεργασία καθώς υπάρχει μεγάλη πιθανότητα παραποίησης των στοιχείων του κάθε αισθητήρα, γεγονός που επηρεάζει κάθε περαιτέρω ανάλυση.

II. Feature Engineering

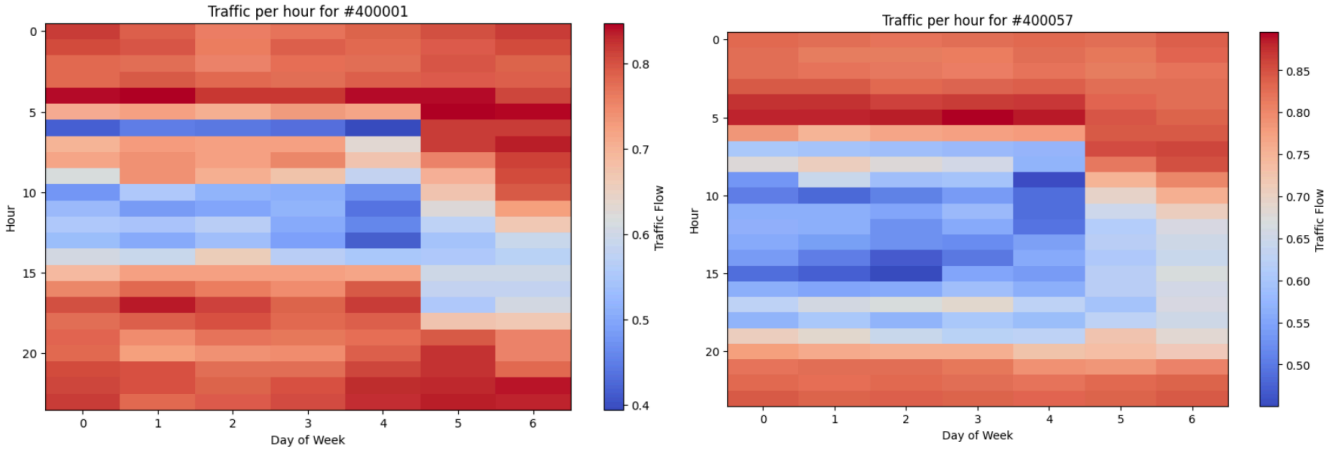
Στο συγκεκριμένο βήμα, κατασκευάσαμε νέα χαρακτηριστικά στο dataframe με σκοπό να εξάγουμε παραπάνω πληροφορία για τις μελλοντικές ενδεχόμενες αναλύσεις. Αρχικά, με την χρήση της βιβλιοθήκης pandas καταφέραμε να δημιουργήσουμε μια στήλη που δείχνει την ώρα (τιμές από 1 έως 23), μια στήλη που δείχνει την ημέρα της εβδομάδας στην οποία έγινε η εγγραφή αναπαριστόντας την αριθμητικά (Δευτέρα - 0, Τρίτη - 1, Τετάρτη - 2, κ.ο.κ.), μια στήλη που δείχνει την εβδομάδα του χρόνου και μια στήλη που δείχνει τον μήνα της εγγραφής. Έπειτα με την χρήση της βιβλιοθήκης holidays καταφέραμε και σημειώσαμε σε boolean μεταβλητή εάν η εκάστοτε μέρα θεωρείται εθνική εορτή στις ΗΠΑ για το έτος 2017. Ανάλογα με το ζητούμενο πρόβλεψης στο τελευταίο ερώτημα μπορούμε να κάνουμε resample τα δεδομένα και να τα φέρουμε σε μορφή ωριαίων, ημερήσιων, εβδομαδιαίων ή και μηνιαίων καταγραφών. Η ετήσια αναδιάταξη των δεδομένων δεν βοηθά σε κάποια ανάλυση αφού τα δεδομένα μας είναι για το μισό έτος 2017.

III. Trend/Seasonal Behavior

Σημείωση: Ο κώδικας για τις συναρτήσεις βρίσκεται στο τμήμα "Παράρτημα κώδικα" του εγγράφου ενώ παραπάνω plot από αυτά που βρίσκονται στο έγγραφο βρίσκονται στον αντίστοιχο φάκελο του παραδοτέου.

Στο συγκεκριμένο βήμα της εργασίας κληθήκαμε να ανακαλύψουμε τυχόν μοτίβα στην συμπεριφορά των δεδομένων. Αυτό έγινε με την χρήση των αντίστοιχων βιβλιοθηκών (matplotlib & seaborn) για την δημιουργία plot. Δημιουργήσαμε 4 διαφορετικά dataframe κάναμε resample τα δεδομένα που είχαμε από τα πεντάλεπτα timestamp σε ωριαία, ημερήσια, εβδομαδιαία και μηνιαία timestamp. Επειδή η απεικόνιση των datapoint για 325 αισθητήρες είναι δύσκολη να γίνει σε ένα plot αποφασίσαμε να κάνουμε την απεικόνιση μεμονωμένα για κάποιους αισθητήρες. Δημιουργήσαμε συναρτήσεις οι οποίες δέχονται ως παράμετρο το sensor_id του loop detector που θέλουμε να απεικονίσουμε και επιστρέφουν το αντίστοιχο plot. Αρχικά, με την χρήση heatmap μπορέσαμε να

απεικονίσουμε την κίνηση που “διαβάζει” ένας loop detector καθόλη την διάρκεια της ημέρας για κάθε μέρα της εβδομάδας σε όλο το διάστημα των 6 μηνών.



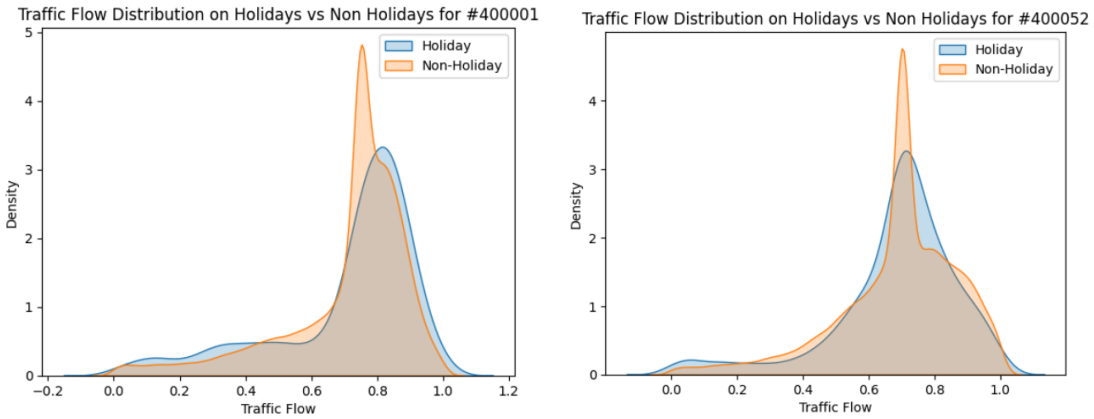
Εικόνα 6 | heatmap για loop detectors 400001 & 400057

Όπως είναι ευδιάκριτο, στην περιοχή που υπάρχει ο πρώτος loop detector (400001) καταγράφηκε περισσότερη κίνηση καθόλη την διάρκεια της καταμέτρησης με μια μείωση στα νούμερα αυτής τις καθημερινές ημέρες (0-4) στις 06:00 & στο διάστημα 10:00-14:00. Αντίστοιχα, στο heatmap του δεύτερου loop detector (400057) βλέπουμε ότι στο χρονικό διάστημα 09:00-19:00 σημειώθηκαν τα χαμηλότερα επίπεδα κίνησης στην συγκεκριμένη περιοχή πάλι τις καθημερινές ημέρες της εβδομάδας.

Αφού πειραματιστήκαμε με τα heatmap σε αρκετούς loop detector παρατηρήσαμε κάποια κοινή συμπεριφορά των δεδομένων.

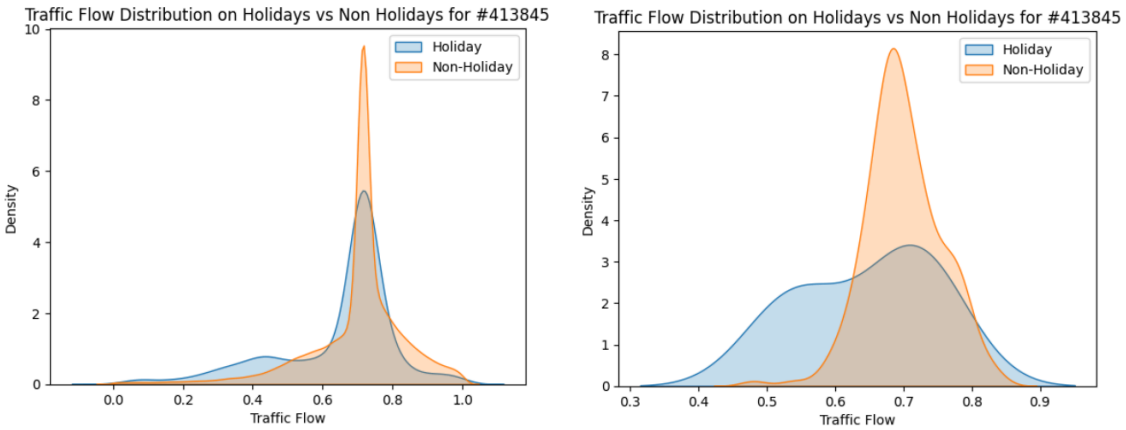
- Στο ξημέρωμα της κάθε ημέρας (ώρες 00:00-05:00) αλλά και το βράδυ (20:00-23:00) υπήρχε αυξημένη κίνηση στους δρόμους.
- Στα χρονικά διαστήματα των καθημερινών ημερών (10:00-14:00) όπου υπήρχε μειωμένη κίνηση υπήρχε μικρή αύξηση τα σαββατοκύριακα (του ύψους 5-8%). Αυτό είναι λογικό καθώς τα σαββατοκύριακα τις πρωινές ώρες (που ο κόσμος συνήθως βρίσκεται στον χώρο εργασίας του τις καθημερινές) δεν δουλεύει και κατ’ επέκτασιν κινείται στους δρόμους.

Με την χρήση των Kernel Density Estimate (KDE) plot, μπορούμε να προβάλουμε την διαφορά σε κίνηση που καταγράφει ένας loop detector τις ημέρες που θεωρούνται αργίες συγκριτικά με τις μη-αργίες. Στην αντίστοιχη συνάρτηση, πέρα από το sensor_id έχουμε ορίσει ως παράμετρο και το dataframe καθώς παίρνουμε διαφορετικά αποτελέσματα από τα κανονικά δεδομένα, τα ωριαία και τα ημερήσια resample.



Εικόνα 7 | KDE plot για loop detectors 400001 & 400052

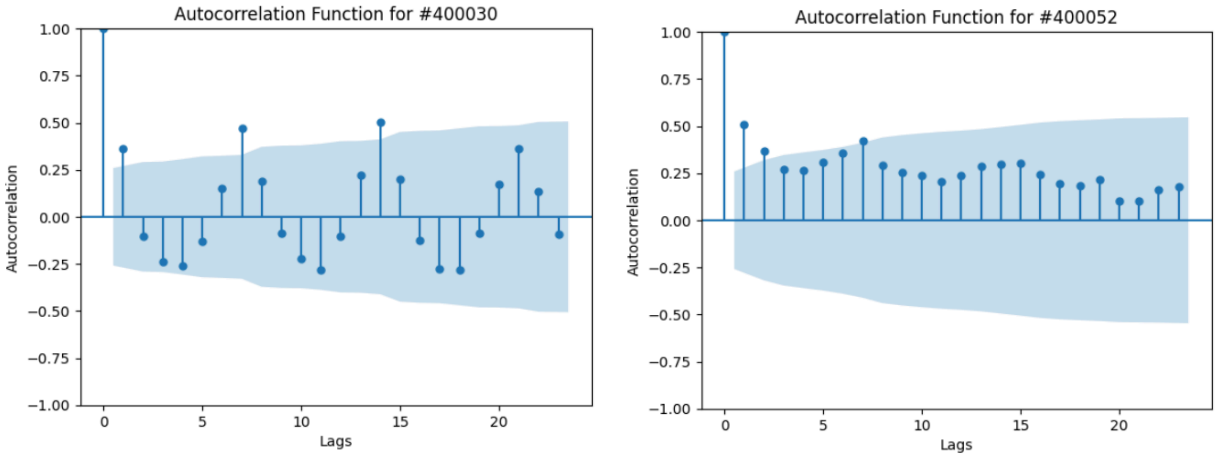
Το συμπέρασμα που εκλάβαμε από τα περισσότερα plot είναι το ίδιο με αυτό που είναι ευδιάκριτο από τα 2 παραπάνω plot. Τα peak των τιμών που καταγράφηκαν στις ημερομηνίες που δεν θεωρούνται αργίες είναι ψηλότερα από αυτά των αργίων. Αυτό είναι λογικό να συμβεί από την στιγμή που περισσότερες ημέρες του εξαμήνου που έχουμε στην διάθεσή μας δεν είναι αργίες άρα υπάρχουν περισσότερα δεδομένα στην κατηγορία 'Non-Holiday', άρα και μεγαλύτερη πιθανότητα πυκνότητας (probability density) στην εμφάνιση των τιμών αυτών. Η εφαρμογή των KDE plot μπορεί να γίνει πέρα από το αρχικό dataframe και στα resampled dataframe με τις ωριαίες/ημερήσιες καταγραφές. Αυτό που παρατηρούμε με την αναπαράσταση των resampled data είναι πως στις ημερήσιες καταγραφές σημειώνεται μεγαλύτερη κοιλάδα (valley) στο plot. Από αυτό συμπεράνουμε πως με την χρήση του daily resampled dataframe έχουμε μια πιο καθαρή και ξεκάθαρη απεικόνιση της διαφοράς στην κίνηση ανάμεσα σε αργίες και μη αργίες.



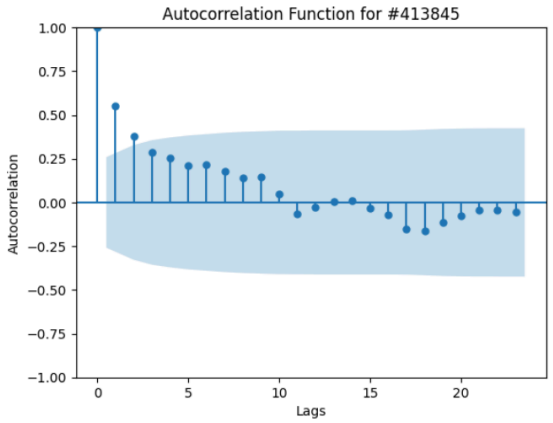
Εικόνα 8 | KDE plot original (αριστερά) & daily (δεξιά) dataframe

Επίσης χρησιμοποιήθηκαν και άλλου είδους plot όπως boxplot και violin plot τα οποία μας οδήγησαν στο συμπέρασμα ότι τις αργίες σημειωνόταν περισσότερη κίνηση. Τέλος, με την

χρήση Autocorrelation Function μπορούμε να πάρουμε πληροφορίες για τα δεδομένα μιας χρονοσειράς. Στην συγκεκριμένη συνάρτηση, για κάποιους loop detector παρατηρήσαμε πως υπάρχει θετική συσχέτιση των δεδομένων της χρονοσειράς καθώς όλα τα point βρίσκονταν πάνω από τον άξονα xx'. Αντίθετα για άλλους loop detector σημειώθηκε σταδιακή μείωση της συσχέτισης καθώς το κάθε point ερχόταν όλο και πιο κοντά στον άξονα, ενώ για άλλους αισθητήρες δεν υπήρχε σταθερή πορεία της συσχέτισης.



Εικόνα 9 | ACF για loop detectors 400030 & 400052

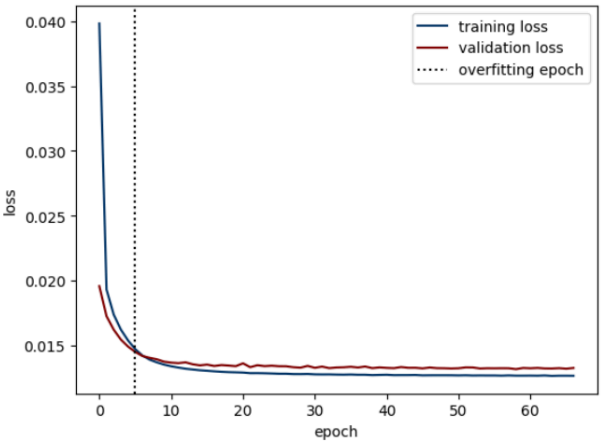


Εικόνα 10 | ACF για loop detector 413845

IV. Forecasting Model

Στο τελευταίο ερώτημα της εργασίας, κληθήκαμε να δημιουργήσουμε ένα μοντέλο πρόβλεψης ταχύτητας κίνησης ανά αισθητήρα για την επόμενη ώρα. Το μοντέλο που επιλέξαμε να δημιουργήσουμε είναι ένα μοντέλο Long Short Term Memory (LSTM) της βιβλιοθήκης TensorFlow. Αρχικά, σπάσαμε το dataset σε 3 μικρότερα για την εκπαίδευση, επαλήθευση και δοκιμή του μοντέλου μας. Αφού χωρίσαμε σαν string μεταβλητές τα

διαστήματα των 3 dataset κάναμε την διάσπαση και ταυτόχρονα αφαιρέσαμε την ιδιότητα του index από τα timestamp. Στην συνέχεια, αφού έγινε έλεγχος για ενδεχόμενα σφάλματα ορίσαμε τα dataframe X & y για κάθε dataset (X_train, y_train, X_val, y_val, X_test, y_test). Τα dataframe X & y περιέχουν τα χαρακτηριστικά που θα χρησιμοποιηθούν στα 3 στάδια δημιουργίας του μοντέλου (training, validation and testing), ενώ το y_test είναι αυτό που θα χρησιμοποιηθεί για την αξιολόγηση του μοντέλου με τις κατάλληλες μετρικές. Για να μπορέσουμε να χρησιμοποιήσουμε τα dataframe στο LSTM μοντέλο πρέπει πρώτα να μετατραπούν σε numpy arrays με την χρήση της αντίστοιχης βιβλιοθήκης. Για να μπορέσουμε να υλοποιήσουμε ένα σωστό και λειτουργικό μοντέλο μηχανικής μάθησης πρέπει να οριστούν σωστά οι παράμετροί του. Συγκεκριμένα ως units για το LSTM μετά από δοκιμές αρκετών νούμερων καταλήξαμε στο 75, ενώ για τα units του Dense βάλαμε 325 (όσοι και οι loop detectors). Επίσης ως εποχές (epochs) εκπαίδευσης ορίσαμε τις 100. Παράλληλα, χρησιμοποιήθηκε η early stopping βιβλιοθήκη από το tensorflow για να σταματά η εκπαίδευση όταν σημειωθούν 8 εποχές χωρίς βελτίωση στο validation loss. Για την αξιολόγηση της επίδοσης του μοντέλου χρησιμοποιήθηκαν οι μετρικές MSE, RMSE, MAPE και R^2. Όπως φαίνεται στο παρακάτω line plot το μοντέλο αρχικά εμφανίζει δυσκολία στην εξοικείωση με το set εκπαίδευσης καθώς σημειώνεται μεγάλη απόκλιση η συγκεκριμένη τιμή από την πρώτη εποχή στη δεύτερη (0.0380 - 0.0194), σφάλμα το οποίο εμφανιζόταν όσες τροποποιήσεις και αν κάναμε στην δομή του μοντέλου και ενδεχομένως να οφείλεται σε πιθανά λάθη προεπεξεργασίας. Από αυτό το σημείο και έπειτα, το μοντέλο λειτουργεί καλά με μοναδικό σημείο overfitting την πέμπτη εποχή εκπαίδευσης.



Εικόνα 11 | LSTM model plot

Για την αξιολόγηση του μοντέλου χρησιμοποιήθηκαν συγκεκριμένες μετρικές οι οποίες καταγράφονται σε ένα dataframe. Αρχικά, ορίσαμε το dataframe με την κατάλληλη ονομασία και έπειτα μέσα σε μια δομή επανάληψης 60 επαναλήψεων (μία για κάθε λεπτό της ώρας) τοποθετούσαμε την αντίστοιχη μετρική λάθους για τα δεδομένα που αφορούν το συγκεκριμένο διάστημα των 5 λεπτών. Ξεκινώντας την δομή επανάληψης από το 0 ορίζουμε ως αρχικό point το 0 και ως τελικό το 12 (αφού θέλουμε να σπάσουμε την μία ώρα σε 12 διαστήματα των 5 λεπτών) και ως τελικό point το 12. Στην επόμενη επανάληψη ως αρχικό point είναι το 13 και ως τελικό το 24. Δηλαδή σε σχέση με την τρεχούμενη μεταβλητή i της δομής ορίζονται ως εξής: αρχικό point = i*12, τελικό point = (i+1)*12. Έπειτα, σε κάθε επανάληψη τοποθετούμε σε μια νέα σειρά/εγγραφή τις 4 μετρικές που ζητούνται δημιουργώντας έτσι ένα dataframe με 5 στήλες (1 index + 4 metrics) και 60 γραμμές. Με την χρήση της groupby μεθόδου μπορέσαμε και αλλάξαμε το πλήθος των σειρών από 60 σε 12. Κάθε μια από τις 12 εγγραφές περιέχει την μέση τιμή 5 εγγραφών του αρχικού dataframe (δηλαδή η πρώτη εγγραφή είναι η μέση τιμή των πρώτων 5 του αρχικού, η δεύτερη εγγραφή είναι η μέση τιμή της δεύτερης πεντάδας κ.ο.κ). Τέλος, για ευκολότερη ανάγνωση του dataframe χρησιμοποιήσαμε ένα dictionary με το οποίο μετατρέψαμε τα αριθμητικά index σε τέτοια που επισημαίνουν το πεντάλεπτο mark.

	MSE	RMSE	MAPE	R2
5 min	0.009659	0.098283	1.250477e-01	-7.298756e+26
10 min	0.018878	0.137396	2.418371e+12	-1.301174e+28
15 min	0.018982	0.137775	3.199298e+12	-1.352662e+28
20 min	0.021374	0.146197	2.943896e+12	-4.468842e+28
25 min	0.010195	0.100969	1.230658e-01	-6.712386e+26
30 min	0.010164	0.100818	6.461841e+11	-5.302950e+26
35 min	0.021336	0.146069	2.656676e+12	-1.242873e+28
40 min	0.018557	0.136223	2.053261e+12	-3.502569e+27
45 min	0.020414	0.142878	2.250283e+12	-6.807192e+27
50 min	0.010595	0.102933	1.227952e-01	-2.191876e+26
55 min	0.011487	0.107177	5.931399e+11	-1.736342e+26
60 min	0.019421	0.139359	4.565025e+12	-2.889319e+28

Εικόνα 12 | metric dataframe

V. Παράρτημα κώδικα

```
In [6]: #outlier detection using Turkey's fences (IQR with threshold)
        thres=1.5 #standard threshold

        #creating a datafram with the outliers using the same index
        df_outliers = pd.DataFrame(index=df.index)

        for sensor in df.columns:
            #calculating the quantiles
            q1 = df[sensor].quantile(0.25)
            q3 = df[sensor].quantile(0.75)
            IQR = q3 - q1

            #calculating the boundaries
            lower = q1 - (thres * IQR)
            upper = q1 + (thres * IQR)

            #if the value is considered an outlier the cell will have the value True
            df_outliers[sensor] = (df[sensor] < lower) | (df[sensor] > upper)
        df_outliers.head()
```

Εικόνα 13 | outlier detection (0-data-preprocess.ipynb)

```
In [11]: from sklearn.preprocessing import MinMaxScaler

         #extracting the columns that need normalizing
         norm_col = new_df.select_dtypes(include='float64').columns
         #extracting the timestamp
         timestamp_idx = new_df.index
         scaler = MinMaxScaler()
         new_df[norm_col] = scaler.fit_transform(new_df[norm_col])
         new_df.index = timestamp_idx

         new_df.head()
```

Εικόνα 14 | data normalization (0-data-preprocess.ipynb)

```
In [11]: #creating features based on the hour, dayofweek
         df['hour']=df.index.hour
         df['dayofweek']=df.index.dayofweek
         df['week']=df.index.isocalendar().week
         df['month']=df.index.month

         df.tail()
```

```
In [12]: #creating feature to determine holiday dates
         df['is_holiday']=df.index.to_series().apply(lambda x: x in holidays.UnitedStates(years=2017))
         df.head()
```

Εικόνα 15 | feature engineering (1-feature-engineering.ipynb)

```
In [58]: df_hourly = df.resample('H').mean()
         df_hourly.index=pd.to_datetime(df_hourly.index)
         df_daily = df.resample('D').mean()
         df_daily.index=pd.to_datetime(df_daily.index)
         df_weekly = df.resample('W').mean()
         df_weekly.index=pd.to_datetime(df_weekly.index)
         df_monthly = df.resample('M').mean()
         df_monthly.index=pd.to_datetime(df_monthly.index)
```

Εικόνα 16 | data resampling (2-data-behavior.ipynb)

```
In [64]: def plot_heatmap(sensor_id):
             '''
                 Args: sensor_id: string, id of the loop detector we plot
                 Returns:  heatmap of the traffic per hour of each day of the week
             '''
             #pivot table for heatmap
             heatdata = df.pivot_table(values=sensor_id, index=df.index.hour, columns=df.index.dayofweek)
             #plot
             plt.figure(figsize=(10,6))
             cax = plt.imshow(heatdata, cmap='coolwarm', aspect='auto')
             plt.colorbar(cax, label='Traffic Flow')
             plt.title('Traffic per hour for #'+sensor_id)
             plt.ylabel('Hour')
             plt.xlabel('Day of Week')
             plt.show()
```

Εικόνα 17 | heatmap function (2-data-behavior.ipynb)

```
In [73]: def plotacf(sensor_id):
             '''
                 Args: sensor_id:string, id the loop detector we plot
                 Returns: Autocorrelation plot
             '''
             plt.figure(figsize=(12, 6))
             acf_plot = plot_acf(df_daily[sensor_id], alpha=0.0005)
             plt.title('Autocorrelation Function for #'+sensor_id)
             plt.xlabel('Lags')
             plt.ylabel('Autocorrelation')
             plt.show()
```

Εικόνα 18 | ACF function (2-data-behavior.ipynb)

```
In [77]: def plot_kde(sensor_id, df):
             '''
                 Args: sensor_id : string, id of the loop detector we want to plot
                       df: dataframe, either the stantard dataframe, the hourly or the daily
                 Returns: Kernel Density Estimate Plot
             '''
             sns.kdeplot(df[df['is_holiday'] == True][sensor_id], label = 'Holiday', fill=True)
             sns.kdeplot(df[df['is_holiday'] == False][sensor_id], label = 'Non-Holiday', fill=True)
             plt.title('Traffic Flow Distribution on Holidays vs Non Holidays for #'+sensor_id)
             plt.ylabel('Density')
             plt.xlabel('Traffic Flow')
             plt.legend()
             plt.show()
```

Εικόνα 19 | KDE function (2-data-behavior.ipynb)

```
#Index reset for train-test split
trainset_no_idx = trainset.reset_index()
valset_no_idx = valset.reset_index()
testset_no_idx = testset.reset_index()

#train, validation and test splits
'''
    using if-else statement in order to catch potential errors
'''
if 'timestamp' in trainset_no_idx.columns:
    '''
        Using if statement for potential errors
    '''
    X_train, y_train = trainset_no_idx.drop(columns=['timestamp']),  trainset_no_idx.drop(columns=['timestamp'])
    X_val, y_val = valset_no_idx.drop(columns=['timestamp']),  valset_no_idx.drop(columns=['timestamp'])
    X_test, y_test = testset_no_idx.drop(columns=['timestamp']),  testset_no_idx.drop(columns=['timestamp'])

     #Reshaping data for LSTM
    X_train = np.reshape(X_train.values, (X_train.shape[0], 1, X_train.shape[1]))
    X_val = np.reshape(X_val.values, (X_val.shape[0], 1, X_val.shape[1]))
    X_test = np.reshape(X_test.values, (X_test.shape[0], 1, X_test.shape[1]))
else:
    print("Timestamp column not found")
```

Εικόνα 20 | training, validation and testing sets (3-data-forecast.ipynb)

```
In [172]: #Defining LSTM model
'''
    LSTM units were initially set to 100 and after adjustments are finally set to 75
    Dense units are set at 325 so that we get output predictions for all loop detectors
'''
model = Sequential()
model.add(LSTM(units=75, activation='relu', input_shape=(X_train.shape[1], X_train.shape[2])))
model.add(Dense(units=325))
model.compile(optimizer='adam', loss='mse')

#model training & early stopping
stop = EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)
mdl = model.fit(X_train, y_train, epochs=100, batch_size=64, validation_data=(X_val, y_val), callbacks=[stop])
```

Εικόνα 21 | LSTM model (3-data-forecast.ipynb)

VI. Βιβλιογραφία

1. Joel Grus, “ΕΠΙΣΤΗΜΗ ΔΕΔΟΜΕΝΩΝ ΒΑΣΙΚΕΣ ΑΡΧΕΣ ΚΑΙ ΕΦΑΡΜΟΓΕΣ ΜΕ PYTHON”, Επιστημονική Ερμηνεία Αριστείδης Θ. Παγουρτζής ΕΜΠ, εκδόσεις Παπασωτηρίου, 2η έκδοση, Αθήνα 2021.
2. Pandas library documentation 2.2 (stable), εδώ και εδώ
3. Matplotlib library documentation 3.8 (stable), εδώ
4. Seaborn library documentation, εδώ και εδώ
5. TensorFlow Keras documentation, εδώ και εδώ